

Linear congruent method for generating random numbers

SM Anis(Nahian)

January 2025

1 Algorithm

The linear congruential method is the easiest way to generate random numbers. If we have a seed number r_i , then to obtain the next random number, we can write the following.

$$r_{i+1} = \text{remainder}\left(\frac{ar_i + c}{M}\right).$$

where a , c , and M are constants.

By the term remainder, we mean the remainder we get from the ratio of $ar_i + c$ and M . The remainder will be the first random number; we will then use this number as r_i to obtain the next random number, and so on. The sequence will be of length M . The sequence of numbers will be over the interval $[0, M-1]$. If we divide r 's by M , the sequence is still of length M , but is no longer a sequence of integers. The interval sequence becomes $[0, 1]$. Moreover, if we want random numbers over the interval $[A, B]$, we can easily determine the following equation by logical thinking.

$$x_i = A + (B - A)r_i, 0 \leq r_i \leq 1$$

2 Codes implemented in Python(for generation one dimensional numbers)

#This is the program for implementation of linear congruent method

```
from matplotlib import pyplot as plt
import numpy as np
ri=float(input('input the seed number'))
a=float(input('input the a'))
c=float(input('input the c'))
M=float(input('input the M'))
#a,c,M are taken so that a non-zero residue is got from the equation
k=int(input('Number of iteration'))
print("The sequence is")
A=float(input('input the A'))
B=float(input('input the B'))
x=[]
y=[]
for i in range(1,(k+1),1):
    #print((ri)/M)
    ri1=((a*ri)+c)%M
    xi=(A+((B-A)*(ri1/M)))    #deviding ri1 with M gives numbers in range[0,1]
    print(xi)

    y.append(xi)
    x.append(i)
    ri=ri1
plt.plot(x,y,'o')
```

```
plt.xlabel('i-')
plt.ylabel('ith-random-number')
plt.savefig("Random-numbers.png", dpi=300)
plt.show()
```

2.1 Results

We take the following inputs to run the program

a=20

c=25

M=103

k=103

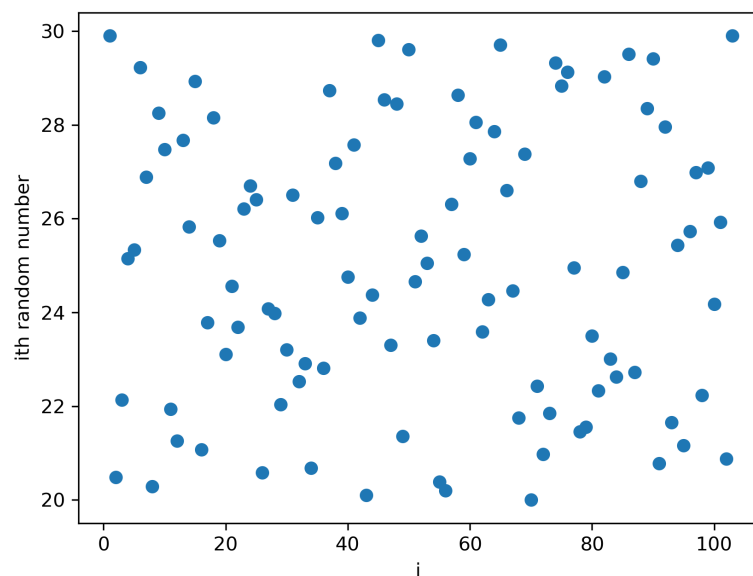


Figure 1: Random numbers generated using the linear congruential method

2.2 Discussion

Observing Figure 1, it can be concluded that there is no pattern in the numbers. So the random number generator has successfully generated random numbers, and the input values are well chosen

3 Codes implemented in Python(for generating two dimensional numbers)

In this case, we use the previous algorithm for random number generation with some addition. If the random numbers on x axis is r_i then the numbers on the y-axis would be $2r_{i-1}$. Taking these things into account, we write the following program

#This is the program for implementation of linear congruent method

```
from matplotlib import pyplot as plt
import numpy as np
ri=float(input('input-the-seed-number'))
```

```

a=float(input('input the a'))
c=float(input('input the c'))
M=float(input('input the M if a,b,c are even then M must be odd'))
k=int(input('Number of iteration'))
print("The sequence is")
A=float(input('input the A'))
B=float(input('input the B'))
x=[]
y=[]
for i in range(1,(k+1),1): #Loop———— (1)
    #print((ri)/M)
    ri1=((a*ri)+c)%M
    xi=(A+((B-A)*(ri1/M)))    #deviding ri1 with M gives numbers in range[0,1]

    x.append(xi)

    ri=ri1

for j in range(0,50,1): #Loop———— (2)
    y.append(x[(2*j)-1])

#As the loop 1 will generate k numbers for x axis. Now we
#want the numbers of the y-axis to be  $y_i = x_{2i} - 1$ . So we can take only less than half
#random numbers of the x-axis for y. That is why we took the upper limit of the loop 2
#to be 50

x= x[:50] #only first 50 numbers will remain in the list
y= y[:50] ##only first 50 numbers will remain in the list
print('x=',x)
print('y=',y)

plt.plot(x,y,'o',label='Random numbers in 2d')
plt.xlabel('x')
plt.ylabel('y')
plt.legend(loc="upper right")
plt.savefig("Random numbers 2d.png", dpi=600)
plt.show()

#r_i=15,a=20,c=25,M=103,K=103,A=15,B=25

```

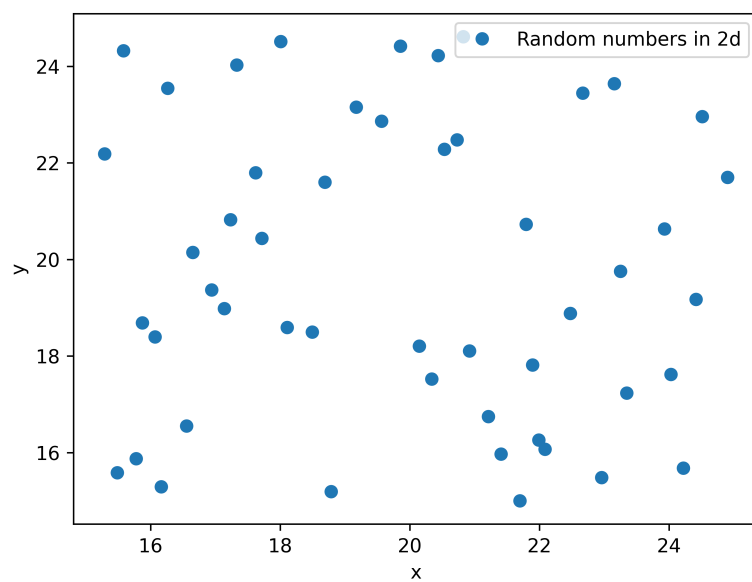
3.1 Results

We take the following inputs to run the program seed number= 15 , a=20 ,c=25 ,M=103,k=103,A=15,B=25

3.2 Discussion

Observing Figure 2, it can be concluded that there is no pattern in the numbers. So the random number generator has successfully generated random numbers, and the input values are well chosen

Reference: Computational Physics: Problem Solving with Computers by Rubin H. Landau (Author), Manuel J PÃ¡jez (Author), Cristian C. Bordeianu (Author)



caption Random numbers
generated using the linear congruential method in 2D